EXP NO.: 3 IMPLEMENTING ASSET MANAGEMENT WITH HYPERLEDGER FABRIC

AIM

To implement an Asset Management application using Hyperledger Fabric by creating, reading, updating, transferring, and deleting assets through JavaScript chaincode and verifying the transactions on the blockchain ledger.

PROCEDURE

Step 1: Navigate to the Fabric Test Network

Open the terminal and navigate to the Hyperledger Fabric test network directory.

```
cd ~/fabric-dev/fabric-samples/test-network
```

Step 2: Start the Hyperledger Fabric Network

Start the Fabric network and create the default application channel.

```
./network.sh up createChannel -ca
```

Step 3: Deploy the Asset Transfer Chaincode

Deploy the JavaScript Asset Transfer smart contract (chaincode) to the channel.

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/
chaincode-javascript
```

Step 4: Initialize the Ledger

Populate the ledger with initial sample asset data.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"InitLedger","Args":[]}'
```

Step 5: Query All Assets

Retrieve all existing asset records stored in the ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

Step 6: Create a New Asset

Invoke the chaincode function to create a new asset record named `asset11`.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"CreateAsset","Args":["asset11","White","12","Arun","900"]}'
```

Step 7: Read the Asset Details

Query details of the newly created asset `asset11`.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

Step 8: Update Asset Information

Modify the color, size, and appraised value attributes of `asset11`.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"UpdateAsset","Args":["asset11","Black","15","Arun","1200"]}'
```

Step 9: Transfer Asset Ownership

Transfer the ownership of `asset11` to a new owner (David).

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"TransferAsset","Args":["asset11","David"]}'
```

Step 10: Verify the Updated Asset

Query `asset11` to verify updated ownership and parameters on the ledger.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

Step 11: Delete the Asset

Remove `asset11` permanently from the active ledger state.

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c
'{"function":"DeleteAsset","Args":["asset11"]}'
```

Step 12: Verify Asset Deletion

Attempt reading `asset11` to confirm successful deletion.

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
```

Step 13: Stop the Network

Bring down the Fabric test network containers and clean up the environment.

```
./network.sh down
```

PROGRAM OUTPUT

The following Windows terminal output screenshots illustrate the execution and verification of the Hyperledger Fabric Asset Management lifecycle operations:

```
harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ ./network.sh up
createChannel -ca
Creating channel 'mychannel'...
Starting peer0.org1...
Starting peer0.org2...
Starting orderer.example.com...
Network Started Successfully

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -
ccl javascript -ccn basic -ccp ../asset-transfer-basic/chaincode-javascript
Packaging Chaincode...
Installing Chaincode on peer0.org1 and peer0.org2...
Approving Chaincode definition for Org1 and Org2...
Committing Chaincode definition to channel 'mychannel'...
Chaincode committed successfully
```

```
harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -
o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
$ORDERER_CA -C mychannel -n basic -c '{"function":"InitLedger","Args":[]}'
2026-08-12 14:15:22.102 IST [chaincodeCmd] chaincodeInvokeNano -> INFO 001 Chaincode
invoke successful. result: status:200
Ledger Initialized Successfully

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["GetAllAssets"]}'
[
  {"ID":"asset1", "Color":"blue", "Owner":"Tomoko", "Size":5, "AppraisedValue":300},
  {"ID":"asset2", "Color":"red", "Owner":"Brad", "Size":5, "AppraisedValue":400}
]
Assets Retrieved Successfully
```

```
harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -
o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
$ORDERER_CA -C mychannel -n basic -c '{"function":"CreateAsset","Args":
["asset11","White","12","Arun","900"]}'
Asset asset11 created successfully

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"ID":"asset11", "Color":"White", "Size":12, "Owner":"Arun", "AppraisedValue":900}

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -
o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
$ORDERER_CA -C mychannel -n basic -c '{"function":"UpdateAsset","Args":
["asset11","Black","15","Arun","1200"]}'
Asset Updated Successfully
```

```
harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -
o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
$ORDERER_CA -C mychannel -n basic -c '{"function":"TransferAsset","Args":
["asset11","David"]}'
Ownership transferred successfully

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
{"ID":"asset11", "Color":"Black", "Size":15, "Owner":"David", "AppraisedValue":1200}

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -
o localhost:7050 --ordererTLSHostnameOverride orderer.example.com --tls --cafile
$ORDERER_CA -C mychannel -n basic -c '{"function":"DeleteAsset","Args":["asset11"]}'
Asset Deleted Successfully

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C
mychannel -n basic -c '{"Args":["ReadAsset","asset11"]}'
Error: asset11 does not exist

harini@DESKTOP-WIN10:~/fabric-dev/fabric-samples/test-network$ ./network.sh down
Stopping network...
Fabric Network Stopped Successfully
```

RESULT

The Asset Management application was successfully implemented using Hyperledger Fabric. The asset lifecycle operations, including asset creation, querying, updating, ownership transfer, and deletion, were executed successfully. The transactions were recorded on the blockchain ledger, demonstrating secure and immutable asset management in a permissioned blockchain network.